CloudSage — AI Design

Runtime AI architecture and development-AI separation

1. Development AI vs Runtime AI

During development, multiple AI coding/reasoning assistants may be used for architecture, implementation, debugging, testing and documentation. These are development tools only.

At runtime, CloudSage uses one selected LLM provider/API key. There is no multi-model or multi-agent requirement for the initial product.

2. AI Input

- Resource metadata
- Cost summaries
- CloudWatch aggregates
- Health/alert state
- Terraform configuration
- Terraform plan
- Relevant historical optimization records
- Environment/criticality context when available

3. AI Input Rules

- No secrets
- No raw credentials
- Bounded context
- Explicit timestamps
- Source labels
- Untrusted text marked as data
- Only information relevant to the decision

4. AI Responsibilities

- Interpret evidence
- Explain optimization rationale
- Identify missing evidence
- Assess qualitative risk
- Suggest a configuration change
- Explain Terraform plan impact
- Produce structured recommendation

5. Non-AI Responsibilities

- Cost arithmetic
- AWS API calls
- Metric aggregation
- Policy enforcement
- Credential handling
- Terraform execution
- Approval enforcement
- Audit persistence
- Safety gates

6. Structured Output

```
{
 "decision": "recommend | reject | needs_more_data",
 "optimization_type": "...",
 "current_configuration": {},
 "recommended_configuration": {},
 "reasoning": [],
 "evidence": [],
 "concerns": [],
 "required_validation": [],
 "risk": "low | medium | high",
 "confidence": 0.0
}
```

7. Tool Calling — Later

If tool calling is introduced, tools should be narrow and read-only first. Write actions remain behind the policy engine and approval system. The model never directly receives arbitrary shell or AWS administrator access.

8. Evaluation

Create a fixed dataset of synthetic/known infrastructure scenarios. Test whether the model recommends downsizing when appropriate, rejects unsafe changes, requests missing data and avoids unsupported claims.

9. Model Provider Abstraction

Keep a provider adapter so one runtime model can be replaced without changing the rest of the application. Only one provider is enabled at any given deployment.

10. RAG — Later

RAG can eventually provide internal runbooks, architecture documentation, previous incidents and optimization policies. It should not be added until the basic structured reasoning pipeline is reliable.